

BLIZKPERSE - ONE-PAGE APP SUMMARY (Repo-based)

Generated from repository evidence on 2026-03-04.

WHAT IT IS

Blizkperse is a chain-agnostic private payout app for stablecoins. Organizers deposit to a shielded pool, and recipients claim with ZK proofs so payout links/amounts are hidden on-chain.

WHO IT'S FOR

Primary persona: organizer/DAO operator who needs private recurring payouts.
Secondary persona: subscriber/recipient who joins organizers and claims funds.

WHAT IT DOES

- Social login + embedded wallets via Para SDK.
- Role-based dashboard: "Organize" and "Receive" paths.
- Organization management: create, rename, delete (with unclaimed-payment guard)
- Recipient subscriptions: browse organizers and join.
- Batch payout creation: select recipients, set amounts, deposit on-chain.
- Persistent data model in PostgreSQL: organizers, subscribers, subscriptions, payouts, payments, notes.
- Claim flow: rebuild Merkle tree, generate proof, register root, withdraw, mark payment claimed.

HOW IT WORKS (ARCHITECTURE)

- Frontend/UI: Next.js App Router pages in `web/app`.
- API layer: Next.js route handlers in `web/app/api` expose data + mutation endpoints.
- Data layer: PostgreSQL via `postgres` client (`web/lib/db.ts`) with runtime schema initialization.
- App state: reactive store (`web/lib/store.ts`) hydrates from `/api/data`, writes through API routes, updates local cache.
- Chain layer: chain context + registry (`web/lib/chain-context.tsx`, `web/lib/constants.ts`) select Monad/Celo RPC + contract addresses.
- Contract layer: viem helpers (`web/lib/contracts.ts`) call ERC-20 + ShieldedPool (approve, deposit, registerRoot, withdraw).
- ZK layer: `web/lib/zk.ts`, `web/lib/merkle.ts`, and `web/app/lib/withdrawProver.ts` compute commitments, build proofs, and generate UltraHonk proof bytes.
- Data flow: User action -> Store/API -> PostgreSQL + chain tx -> deposit events / notes -> proof generation -> withdraw tx -> DB payment status update.

HOW TO RUN (MINIMAL)

1. Prerequisites: Node.js >= 20, `DATABASE\_URL`, `NEXT\_PUBLIC\_PARA\_API\_KEY`.
2. `cd web`
3. `npm install`
4. `cp .env.example .env.local`
5. Set `.env.local` values for `DATABASE\_URL` and `NEXT\_PUBLIC\_PARA\_API\_KEY`.
6. `npm run dev` (uses webpack in the script).
7. Open <http://localhost:3000>

REPO GAPS

- Production-grade authz policy and role enforcement details: Not found in repo.
- Observability/monitoring setup (alerts, dashboards): Not found in repo.